

1 [A.] Explain the various terminology with properties of the below RDBMS.

Emp_Id	E_NAME	POST	salary
E1	Rahul	clerk	20000
E2	Kapil	Manager	80000
E3	Mukesh	Clerk	20000
E4	Manoj	Peon	100000

◆ Table Name (Relation)

In RDBMS, a **table** is called a **relation**. It stores data in rows and columns.

👉 In this case, the table represents **employee details**, so we can name it:
Employee

◆ Columns (Attributes or Fields)

Each column in a table is called an **attribute** and describes a specific property of the entity.

Attribute Name	Description	Data Type
Emp_Id	Unique ID for each employee	VARCHAR/Text
E_NAME	Name of the employee	VARCHAR/Text
POST	Job position	VARCHAR/Text
salary	Monthly salary	INTEGER/Number

✅ Properties of Attributes:

- Emp_Id: **Primary Key** (Uniquely identifies each row)
- salary: Can be used for **aggregations** (e.g., AVG(salary), MAX(salary))

◆ Rows (Tuples or Records)

Each row represents a **record** or **tuple**, containing data about one entity.

Example tuples:

- (E1, Rahul, clerk, 20000)
- (E2, Kapil, Manager, 80000)

✅ Properties:

- Each tuple is **unique**.
- Order of tuples does **not matter** in a relational model.

◆ Primary Key

A **primary key** is an attribute (or set of attributes) that **uniquely identifies each tuple** in the table.

👉 Here, Emp_Id is the **primary key**, since no two employees have the same Emp_Id.

◆ Domain

The **domain** of an attribute is the set of **valid values** it can take.

Examples:

- Emp_Id: E1, E2, E3 (alphanumeric IDs)
 - salary: Whole numbers (positive integers)
-

◆ Cardinality

The number of **rows/tuples** in a table.

👉 Here: **Cardinality = 4** (since there are 4 employees)

◆ Degree

The number of **columns/attributes** in a table.

👉 Here: **Degree = 4** (Emp_Id, E_NAME, POST, salary)

◆ Data Integrity Constraints

These are rules that ensure **accuracy and consistency** of data:

- **Entity Integrity:** Primary key (Emp_Id) must be unique and not null.
 - **Domain Integrity:** Each column must contain values of a specific data type.
 - **Referential Integrity** (if there are foreign keys): Ensures valid references between tables (not applicable here).
-

1.[B] Relational Algebra expression-

(a) Find the ID and name of each instructor in the Physics department

1. $\sigma_{dept_name=Physics}(instructor)$
2. $\pi_{ID,name}(result)$

(b) Find the ID and name of each instructor in a department located in the building "Watson"

1. $\sigma_{building=Watson}(department)$
2. $result \bowtie instructor$
3. $\pi_{ID,name}(result)$

(c) Find the ID and name of each student who has taken at least one course in the "Comp. Sci." department

1. $student \bowtie takes$
2. $result \bowtie course$
3. $\sigma_{course_dept_name=Comp.Sci.}(result)$
4. $\pi_{student.ID,student.name}(result)$

(d) Find the ID and name of each student who has taken at least one course section in the year 2018

1. $student \bowtie takes$
2. $\sigma_{year=2018}(result)$
3. $\pi_{ID,name}(result)$

(e) Find the ID and name of each student who has not taken any course section in the year 2018

1. $A \leftarrow \pi_{ID,name}(student)$
2. $B \leftarrow \pi_{ID,name}(\sigma_{year=2018}(student \bowtie takes))$
3. $A - B$

Compact Version

(a) Find the ID and name of each instructor in the Physics department
 Answer: $\pi_{ID, name}(\sigma_{dept_name=Physics}(instructor))$

(b) Find the ID and name of each instructor in a department located in the building "Watson"

Answer: $\pi_{ID, name}(\sigma_{building='Watson'}(department) \bowtie instructor)$

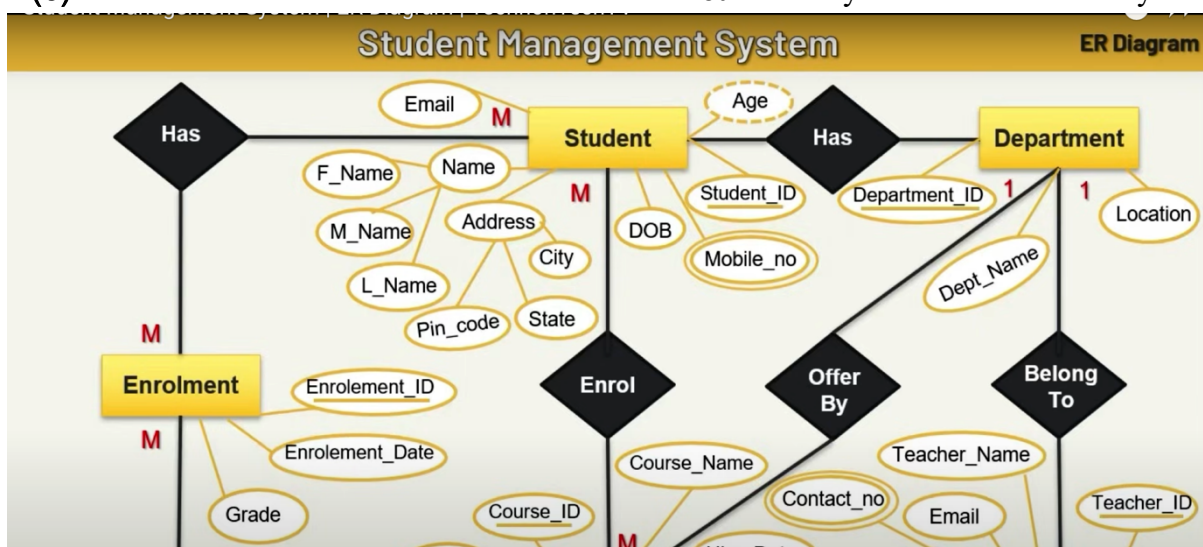
(c) Find the ID and name of each student who has taken at least one course in the "Comp. Sci." department.

Answer: $\pi_{student.ID, student.name}(\sigma_{course.dept_name='Comp. Sci.'}((student \bowtie takes) \bowtie course))$

(d) Find the ID and name of each student who has taken at least one course section in the year 2018.

Answer: $\pi_{ID, name}(\sigma_{year=2018}(student \bowtie takes))$

(e) Find the ID and name of each student who has **not** taken any course section in the year



1.[C]. Consider the foreign-key constraint from the dept_name attribute of instructor to the department relation. Give examples of inserts and deletes to these relations that can cause a violation of the foreign-key constraint.

Answer:

`instructor(dept_name) → department(dept_name)`

This means dept_name in instructor is a **foreign key** that must exist in the department table.

মানে, instructor টেবিলের dept_name ফিল্ডে যেই ভ্যালু থাকবে, সেটা অবশ্যই department টেবিলে থাকতে হবে।

 1. Insert that causes Foreign Key Violation:

Query:

```
INSERT INTO instructor (ID, name, dept_name, salary)
VALUES ('I555', 'Tuhin', 'Robotics', 60000);
```

Reason:

The value 'Robotics' is not present in the department table.

Since instructor.dept_name refers to department.dept_name, this violates the foreign key constraint.

Problem: এখানে dept_name = 'Robotics', কিন্তু department টেবিলে 'Robotics' নামে কোনো department নেই।

 **Foreign key violation** হবে কারণ referenced value নেই।

 2. Delete that causes Foreign Key Violation:

Query:

```
DELETE FROM department
WHERE dept_name = 'Physics';
```

Reason:

If there is any instructor with dept_name = 'Physics', deleting this department will leave that instructor with a non-existent department reference, violating the foreign key constraint.

Problem: যদি instructor টেবিলে এমন কোনো row থাকে যেখানে dept_name = 'Physics', তাহলে department থেকে Physics ডিলিট করলে সেই instructor-এর foreign key আর valid থাকবে না।

🚫 Foreign key violation হবে কারণ referenced row delete হয়ে গেছে।

✅ **How to avoid these violations:**

- Before inserting into instructor, check that the dept_name exists in department.
- Before deleting from department, ensure no instructor is using that dept_name.

2[a] Find the total grade points earned by the student with ID '12345', across all courses taken by the student.

SQL কোড:

```
SELECT SUM(gp.points * t.credits) AS total_grade_points
FROM takes t
JOIN grade_points gp ON t.grade = gp.grade
WHERE t.ID = '12345';
```

ব্যাখ্যা: এই কোডে, আমরা takes টেবিল থেকে ছাত্র '12345' এর সমস্ত কোর্সের তথ্য সংগ্রহ করছি। তারপর, grade_points টেবিলের সাথে যোগ (JOIN) করে প্রতিটি কোর্সের গ্রেডের পয়েন্ট বের করছি। প্রত্যেক কোর্সের পয়েন্ট এবং ক্রেডিট গুণ করে আমরা মোট গ্রেড পয়েন্ট হিসাব করছি। SUM ফাংশন ব্যবহার করে সব কোর্সের গ্রেড পয়েন্ট যোগ করা হচ্ছে, যাতে ছাত্রটির মোট গ্রেড পয়েন্ট পাওয়া যায়।

b. Find the grade point average (GPA) for the above student, that is, the total grade points divided by the total credits for the associated courses.

SQL code:

```
SELECT
SUM(gp.points * t.credits) / SUM(t.credits) AS GPA
```

```
FROM takes t
JOIN grade_points gp ON t.grade = gp.grade
WHERE t.ID = '12345';
```

ব্যাখ্যা: এখানে আমরা GPA বের করতে চাই। GPA বের করার জন্য, মোট গ্রেড পয়েন্ট (যা গুণফল করে যোগ করা হয়) মোট ক্রেডিট দ্বারা ভাগ করতে হবে। কোডে, প্রথমে গ্রেড পয়েন্ট এবং ক্রেডিট গুণফল নিয়ে তাদের যোগফল বের করা হয়েছে। তারপর, SUM(t.credits) দিয়ে মোট ক্রেডিট যোগ করে, মোট গ্রেড পয়েন্টকে মোট ক্রেডিট দ্বারা ভাগ করা হচ্ছে, যা GPA হিসাব করবে।

c. Find the ID and the grade-point average of each student.

SQL code:

```
SELECT
    t.ID,
    SUM(gp.points * t.credits) / SUM(t.credits) AS GPA
FROM takes t
JOIN grade_points gp ON t.grade = gp.grade
GROUP BY t.ID;
```

ব্যাখ্যা: এখানে আমরা প্রতিটি ছাত্রের GPA বের করতে চাই। GROUP BY ব্যবহার করা হয়েছে যাতে প্রত্যেক ছাত্রের আলাদা আলাদা GPA হিসাব করা যায়। প্রতিটি ছাত্রের জন্য তাদের সব কোর্সের গ্রেড পয়েন্ট এবং ক্রেডিট গুণফল করে মোট গ্রেড পয়েন্ট বের করা হচ্ছে। তারপর, মোট গ্রেড পয়েন্টকে মোট ক্রেডিট দিয়ে ভাগ করে GPA বের করা হচ্ছে। এই কোডটি সব ছাত্রের জন্য GPA হিসাব করবে এবং তাদের ID সহ ফলাফল দেখাবে।

d. Insert every student whose tot cred attribute is greater than 100 as an instructor in the same denartment. with a salary of 10,000 taka.

SQL কোড:

```
INSERT INTO instructor (ID, name, dept_name, salary)
SELECT s.ID, s.name, s.dept_name, 10000
FROM student s
WHERE s.tot_cred > 100;
```

ব্যাখ্যা: এই কোডে, আমরা student টেবিল থেকে ছাত্রদের খুঁজে বের করছি যাদের মোট ক্রেডিট (tot_cred) ১০০ এর বেশি। এই ছাত্রদের instructor টেবিলে ইনসার্ট (insert) করা হচ্ছে। তাদের ID, নাম, এবং বিভাগ নাম সাথে স্যালারি ১০,০০০ টাকাও দেওয়া হচ্ছে। এটি INSERT INTO কমান্ডের মাধ্যমে করা হচ্ছে, যেখানে সিলেক্ট করা ছাত্রদের তথ্য নতুন ইনস্ট্রাকটর হিসেবে টেবিলে যোগ করা হচ্ছে।

[B.]the SQL like operator is case sensitice (in most systems), but the lower function on strings can be used to perform case-insensitive matching. To show how, write a query that finds departments whose names contain the string "sci" as a substring, regardless of the case.

এটি এমন একটি প্রশ্ন যেখানে আপনাকে SQL কোড লেখার জন্য বলা হয়েছে, যাতে "sci" শব্দটি কোনো ডিপার্টমেন্টের নাম-এ (যে নামেই "sci" আছে) কেস-ইনসেনসিটিভভাবে অনুসন্ধান করা যায়।

```
SELECT dept_name
FROM department
WHERE LOWER(dept_name) LIKE '%sci%';
```

অবশ্যই, Mehedi sir! চলুন প্রশ্নটা বাংলা ভাষায় সহজ করে বুঝে নিই:

2[B]. Consider the SQL query:

```
SELECT p.a1
FROM p, r1, r2
WHERE p.a1 = r1.a1 OR p.a1 = r2.a1;
```

এই SQL কোডটি বলছে:

➡ এমন p.a1 গুলো বের করো, যেগুলো r1.a1 অথবা r2.a1 এর সমান।

◆ তাহলে সমস্যা কোথায়?

এই অংশটা লক্ষ করো:

```
FROM p, r1, r2
```

এখানে **তিনটা টেবিল** একসাথে লেখা মানে হচ্ছে **CROSS JOIN**। মানে হচ্ছে:

- p এর প্রতিটি row
- r1 এর প্রতিটি row
- r2 এর প্রতিটি row

তাদের **গুনফল** (কার্টেসিয়ান প্রোডাক্ট) হবে।

অর্থাৎ:

👉 সব কন্ডিশনেশন বানিয়ে তারপর WHERE চেক করবে।

💡 সমস্যা তখন হয় যখন r_1 বা r_2 খালি হয়:

ধরো r_1 এ কোনো row নাই — অর্থাৎ r_1 empty।

তাহলে:

$$|p \times r_1 \times r_2| = 0$$

কারণ, যদি একটি টেবিল খালি হয়, তাহলে পুরো মিলে কিছুই মিলবে না।

WHERE ক্লজের আগেই টেবিলের রো থাকবে না — মানে output **কিছুই আসবে না**।

✅ তাহলে কী করলে ভালো?

তুমি যদি চাও যে,

- $p.a1$ বের হোক যদি $r1.a1$ অথবা $r2.a1$ তে মেলে,
- এবং r_1 বা r_2 খালি হলেও কাজ করুক,

তাহলে নিচের মত করে EXISTS দিয়ে লেখো:

```
SELECT p.a1
FROM p
WHERE EXISTS (SELECT 1 FROM r1 WHERE r1.a1 = p.a1)
OR EXISTS (SELECT 1 FROM r2 WHERE r2.a1 = p.a1);
```

- ◆ এতে r_1 বা r_2 খালি হলেও সমস্যা হবে না।
- ◆ $p.a1$ মিলে গেলে ঠিকমতো বের হবে।

2[C]. Using the university schema, write an SQL query to find the IDs of those students who have retaken at least three distinct courses at least once (i.e, the student has taken the course at least two times).

এমন student-এর ID বের করতে হবে —

যারা **কমপক্ষে ৩টি ভিন্ন course ২ বার বা তার বেশি বার** নিয়েছে।

অর্থাৎ, একজন student যদি CSE101, MAT102, PHY103 — এই তিনটি course ২ বার বা তার বেশি করে নেয় — তাহলে সে কোয়েরির ফলাফলে থাকবে।

```
SELECT ID
FROM (
    SELECT ID, course_id
    FROM takes
    GROUP BY ID, course_id
    HAVING COUNT(*) >= 2
) AS retaken_courses
GROUP BY ID
HAVING COUNT(course_id) >= 3;
```

◆ ভিতরের অংশ:

```
SELECT ID, course_id
FROM takes
GROUP BY ID, course_id
HAVING COUNT(*) >= 2
```

👉 এটা বের করে কে কোন course ২ বার বা তার বেশি নিয়েছে।

◆ বাইরের অংশ:

```
GROUP BY ID
HAVING COUNT(course_id) >= 3
```

অবশ্যই, Mehedi Sir! নিচে আমি SQL, MySQL, আর SQL Server এর মধ্যে সবচেয়ে গুরুত্বপূর্ণ ৬টি পার্থক্য (important differences) দিচ্ছি, যেগুলো পরীক্ষায় বা ভাইভায় কাজে আসবে:

✅ Top 6 Important Differences (SQL vs MySQL vs SQL Server)

Feature	SQL	MySQL	SQL Server
Type	Query Language	Open-source Database System	Commercial Database System
Developer	ANSI / ISO Standard	Oracle (originally MySQL AB)	Microsoft
Platform	Used in all RDBMS	Cross-platform (Windows/Linux)	Mainly Windows (supports Linux now)
Syntax Extension	Standard SQL	SQL + MySQL-specific extensions	T-SQL (SQL + Procedural extensions)
Security	Not applicable	Medium security	High enterprise-level security
Best Use Case	Writing queries	Web & small apps (e.g., WordPress)	Enterprise apps, banking, analytics

আরও সংক্ষেপে দরকার হলে, mnemonic বা shortcut দিয়ে মনে রাখার ট্রিকও দিতে পারি! বলবেন? 😊

2.[c]comparison between SQL, MySQL, and MS SQL Server:

Feature	SQL	MySQL	MS SQL Server
Developed by	Not by any one company	Developed by Oracle	Developed by Microsoft
Programming Languages Supported	C++, Java, Ruby, Visual Basic, etc.	Perl, Tcl, Haskell, etc.	C++, Java, .NET, etc.
Storage Space Requirement	Varies by the database system	Needs less space	Needs more space
Stopping a Query	Can stop queries	Cannot stop queries midway	Can stop queries
Backup Process	Does not block the database during backup	Blocks the database during backup	Does not block the database during backup
Cost	Not free (depends on the system)	Free (open-source)	Not free (needs a license)
Security	Depends on the system	Allows file manipulation during use	High security, no file manipulation allowed during use
Editions Available	No specific editions	Standard, Enterprise, Cluster Edition	Enterprise, Standard, Web, Workgroup, Express

3.[A] Perhaps the most important data items in any database system are the passwords that control access to the database. Suggest a scheme for the secure storage of passwords. Be sure that your scheme allows the system to test passwords supplied by users who are attempting to log into the system.

পাসওয়ার্ড নিরাপদভাবে সংরক্ষণ করার জন্য আমরা একটি বিশেষ পদ্ধতি ব্যবহার করি, যার মধ্যে **হ্যাশিং** এবং **সল্টিং** দুটি গুরুত্বপূর্ণ অংশ থাকে। এই পদ্ধতির মাধ্যমে আমরা পাসওয়ার্ডগুলো এমনভাবে সংরক্ষণ করি যাতে সেগুলো কোনোভাবেই উদ্ধার করা না যায়।

১. পাসওয়ার্ড হ্যাশিং (Password Hashing):

- **হ্যাশিং** হলো একটি পদ্ধতি যার মাধ্যমে পাসওয়ার্ডের একটি এক্সিকিউটেবল (যে পরিবর্তন করা সম্ভব নয় এমন) ভাঙ্গন তৈরি করা হয়। এই প্রক্রিয়াতে পাসওয়ার্ডটি এক ধরনের কোড বা "হ্যাশ" হিসেবে পরিণত হয় যা আসল পাসওয়ার্ডের প্রতিনিধিত্ব করে।
- উদাহরণস্বরূপ, যদি আপনার পাসওয়ার্ড হয় "mypassword123", তাহলে এটি একটি এক ধরনের কোডে পরিণত হবে, যা শুধুমাত্র সঠিক পাসওয়ার্ড দিয়েই পুনরায় জানা যেতে পারে।

২. সল্টিং (Salting):

- **সল্ট** হলো একটি এলোমেলো বাল্ডম স্ট্রিং যা পাসওয়ার্ডের সঙ্গে যোগ করা হয় হ্যাশিং করার আগে। এর ফলে, এমনকি যদি দুজনের পাসওয়ার্ড এক হয়, তবুও তাদের হ্যাশ আলাদা হবে কারণ সল্ট আলাদা হবে।
- উদাহরণস্বরূপ, আপনি যদি "mypassword123" পাসওয়ার্ডের সঙ্গে "random_salt" সল্ট যোগ করেন এবং এটি হ্যাশ করেন, তাহলে ফলস্বরূপ হ্যাশটি হবে একেবারেই আলাদা।

এমনকি যদি অন্য কারোও একই পাসওয়ার্ড থাকে, কিন্তু সল্ট আলাদা থাকে, তাদের হ্যাশও আলাদা হবে।

৩. পাসওয়ার্ড স্টোর করার পদ্ধতি:

পাসওয়ার্ড সঠিকভাবে সংরক্ষণ করতে:

1. **সল্ট তৈরি করুন:** পাসওয়ার্ড নেওয়ার সময় একটি এলোমেলো সল্ট তৈরি করুন (যেমন ১৬ বাইট দীর্ঘ)।
2. **সল্ট এবং পাসওয়ার্ডকে একত্রে হ্যাশ করুন:** সল্ট এবং পাসওয়ার্ড একত্রে যোগ করুন এবং তারপরে একটি শক্তিশালী হ্যাশিং অ্যালগরিদম (যেমন bcrypt, Argon2) ব্যবহার করে হ্যাশ করুন।
3. **হ্যাশ এবং সল্ট ডাটাবেজে সংরক্ষণ করুন:** এই হ্যাশ এবং সল্টটি ডাটাবেজে সঞ্চয় করুন। সল্টটি প্লেইনটেক্সট হিসাবে রাখা যেতে পারে, কারণ এটি গোপনীয় নয়।

উদাহরণস্বরূপ, একটি টেবিল হবে:

user_id	username	password_hash	salt
1	user1	\$2a\$12\$6u5r3v4r98Pp4JIHJ68p.q6Za27uVwZ9.	random_salt
2	user2	\$2a\$12\$hjgG7H7HsdHGje2g87D.jht92zF9wZZw.	random_salt

৪. পাসওয়ার্ড যাচাই করা (Password Verification):

যখন একজন ব্যবহারকারী লগইন করতে চায়:

1. **সল্ট এবং হ্যাশ পুনরুদ্ধার করুন:** ডাটাবেজ থেকে ব্যবহারকারীর সল্ট এবং হ্যাশ পুনরুদ্ধার করুন।
2. **পাসওয়ার্ড হ্যাশ করুন:** ব্যবহারকারী যে পাসওয়ার্ডটি দিয়েছে তা দিয়ে একই সল্ট ব্যবহার করে হ্যাশ তৈরি করুন।
3. **হ্যাশের সাথে তুলনা করুন:** যদি নতুন করে তৈরি করা হ্যাশটি ডাটাবেজে সংরক্ষিত হ্যাশের সাথে মিলে যায়, তাহলে পাসওয়ার্ড সঠিক, এবং ব্যবহারকারী সফলভাবে লগইন করতে পারে।

উদাহরণ:

1. **ব্যবহারকারী পাসওয়ার্ড তৈরি করল:**
 - পাসওয়ার্ড: mypassword123
 - সল্ট: random_salt
 - হ্যাশিং: `bcrypt('random_salt' + 'mypassword123')`
 - ধরি, এর ফলাফল হলো: 3f6a8bc7a9112...
 - হ্যাশ এবং সল্ট ডাটাবেজে সঞ্চয় করুন।

2. ব্যবহারকারী লগইন করতে চাইল:

- ব্যবহারকারী যে পাসওয়ার্ডটি দিয়েছে: mypassword123
- ডাটাবেজ থেকে সল্ট পাওয়া: random_salt
- হ্যাশ তৈরি: `bcrypt('random_salt' + 'mypassword123')`
- ডাটাবেজের সঞ্চিত হ্যাশের সঙ্গে (3f6a8bc7a9112...) নতুন হ্যাশের (3f6a8bc7a9112...) তুলনা করা হয়। যদি তারা মিলে যায়, তাহলে লগইন সফল।

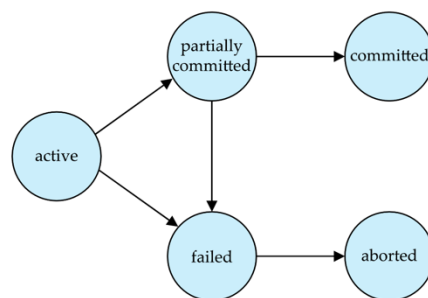
নিরাপত্তা পরামর্শ:

- **শক্তিশালী হ্যাশিং অ্যালগরিদম ব্যবহার করুন:** bcrypt, Argon2 বা PBKDF2 ব্যবহার করুন, কারণ এগুলো পাসওয়ার্ড সুরক্ষার জন্য ডিজাইন করা হয়েছে এবং ব্রুট ফোর্স আক্রমণ থেকে রক্ষা করে।
- **প্রত্যেক পাসওয়ার্ডের জন্য আলাদা সল্ট ব্যবহার করুন:** প্রতিটি ব্যবহারকারীর জন্য আলাদা সল্ট তৈরি করুন।
- **প্লেইনটেক্সট পাসওয়ার্ড সংরক্ষণ করবেন না:** শুধু হ্যাশ এবং সল্ট সংরক্ষণ করুন, কখনো পাসওয়ার্ড প্লেইনটেক্সটে রাখবেন না।
- **শক্তিশালী পাসওয়ার্ড নীতিমালা প্রয়োগ করুন:** ব্যবহারকারীদের শক্তিশালী পাসওয়ার্ড ব্যবহার করতে উৎসাহিত করুন।

এই পদ্ধতি পাসওয়ার্ড সুরক্ষিত রাখে এবং আপনার সিস্টেমে প্রবেশের জন্য অবৈধ চেষ্টা থেকে রক্ষা করে।

3.[B] During its execution, a transaction passes through several states, until it finally commits or aborts. List all possible sequences of states through which a transaction may pass. Explain why each state transition may occur.

Transaction State (Cont.)



State Name	Description
------------	-------------

1. Active:	The transaction is currently executing its operations (read/write).
-------------------	---

2. Partially Committed:	The transaction has completed its final operation but changes are not yet permanently saved.
--------------------------------	--

- 3. **Committed:** All the changes made by the transaction are permanently saved in the database.
- 4. **Failed:** Some error occurred; the transaction cannot proceed.
- 5. **Aborted:** All changes made are undone (rolled back), and the transaction is terminated.

Transaction State Change গুলো সহজে বোঝা

✅ 1. Active → Partially Committed

- 👉 যখন ট্রান্সাকশন (Transaction) সব কাজ (read, write, হিসাব ইত্যাদি) শেষ করে,
- 👉 তখন সে বলে: “আমি সব কাজ শেষ করেছি”
- ⚠️ কিন্তু এখনো ডাটাবেজে স্থায়ীভাবে কিছু লেখা হয়নি।

উদাহরণ: তুমি ফর্ম পূরণ করেছো, এখন সাবমিট বাটনে ক্লিক করছো — কিন্তু এখনো ডাটা জমা হয়নি।

✅ 2. Partially Committed → Committed

- 👉 সবকিছু ঠিকঠাক থাকলে, ডাটা সফলভাবে ডাটাবেজে জমা হয়।
- 👉 এখন এটা স্থায়ী হয়ে গেছে — আর পরিবর্তন করা যাবে না।

উদাহরণ: ফর্মটা সফলভাবে জমা হয়েছে, তুমি কনফার্মেশন মেসেজ পেয়ে গেছো।

❌ 3. Active → Failed

- 👉 কাজ করতে করতে হঠাৎ কোন সমস্যা হল — যেমন:

- বিদ্যুৎ চলে গেলো
 - কোডে bug ছিল
 - ইন্টারনেট চলে গেলো
- তখন ট্রান্সাকশন ব্যর্থ হয়।

উদাহরণ: ফর্ম পূরণ করতে করতে হঠাৎ পেইজ রিলোড হয়ে গেলো।

❌ 4. Partially Committed → Failed

- 👉 সব কাজ শেষ হলেও, ডাটাবেজে জমা হওয়ার আগেই সমস্যা হল, যেমন: সার্ভার হঠাৎ বন্ধ হয়ে গেলো।

উদাহরণ: তুমি সাবমিট করেছো কিন্তু বিদ্যুৎ চলে গেছে — ডাটা জমা হয়নি।

5. Failed → Aborted

👉 যেহেতু কাজ শেষ হয়নি, তাই **সব কিছু বাতিল করা হয়।**

👉 ডাটাবেজ থেকে যেই জিনিসগুলো লেখা হয়েছিলো, সব **রোলব্যাক (undo)** করা হয়।

উদাহরণ: আগের কাজগুলো ভুল ছিলো, তাই সব মুছে ফেলা হলো।

6. Aborted → Active (optional)

👉 কিছু ট্রান্সাকশন আবার **নতুনভাবে শুরু** হতে পারে —
এটা **ম্যানুয়ালি** বা **অটোমেটিক** হতে পারে।

উদাহরণ: তুমি আবার ফর্ম পূরণ শুরু করলে।

3[C]. Explain the transaction property with an example of transaction T1 and T2

ACID প্রপার্টিগুলোর ব্যাখ্যা এই উদাহরণের মাধ্যমে

উদাহরণ: ব্যাংকের দুইটা অ্যাকাউন্ট

- A নামের অ্যাকাউন্টে টাকা আছে: ₹1000
 - B নামের অ্যাকাউন্টে টাকা আছে: ₹2000
-

ট্রানজ্যাকশন T1:

T1: A থেকে B-তে ₹500 ট্রান্সফার করো

1. Read(A) → A = ₹1000
 2. A := A - 500 → A = ₹500
 3. Write(A)
 4. Read(B) → B = ₹2000
 5. B := B + 500 → B = ₹2500
 6. Write(B)
-

💰 ট্রানজ্যাকশন T2:

T2: B-তে ₹200 ইন্টারেস্ট যোগ করো

1. Read(B) → B = ₹2500 (বা পূর্বের মান)
2. B := B + 200 → B = ₹2700
3. Write(B)

✅ ACID প্রপার্টিগুলোর ব্যাখ্যা এই উদাহরণের মাধ্যমে:

1. ✅ Atomicity (অ্যাটমিকিটি)

ব্যাখ্যা:

Everything happens together or nothing happens

T1 যদি মাঝপথে (ধরি ₹500 A থেকে কেটে নিয়েছে, কিন্তু B-তে যোগ করেনি) বন্ধ হয়ে যায়, তাহলে সিস্টেম সেটা বাতিল করবে আর A-তে টাকা ফিরিয়ে দেবে।

2. ✅ Consistency (কনসিসটেন্সি)

ব্যাখ্যা:

Database remains correct before and after transaction

T1 + T2 করলে মোট টাকা:

- শুরুতে: ₹1000 + ₹2000 = ₹3000
- শেষে: ₹500 (A) + ₹2700 (B) = ₹3200 → (₹200 ইন্টারেস্ট যোগ হওয়ায় ঠিক আছে)

3. ✅ Isolation (আইসোলেশন)

One transaction doesn't affect another

ব্যাখ্যা:

দুইটা ট্রানজ্যাকশন একসাথে চললেও, তারা একে অন্যের উপরে প্রভাব ফেলতে পারবে না।

যদি T1 ও T2 একসাথে চলে, তবে B-র ফাইনাল ভ্যালু ভুল হতে পারে যদি Isolation না থাকে।

👉 Isolation নিশ্চিত করে যে: ফলাফল যেন এমন হয়, যেন তারা একে একে চলেছে।

4. ✓ Durability (ডিউরেবিলিটি)

Once committed, it is permanent

ব্যাখ্যা:

ট্রানজ্যাকশন একবার হয়ে গেলে (Committed), সেটা স্থায়ীভাবে ডাটাবেজে থাকবে।

T1 যদি সফল হয় এবং Committed হয়, তাহলে পাওয়ার চলে গেলেও A-তে ₹500 কমে যাবে, আর B-তে ₹500 বাড়বে — এই চেক্স হারাবে না।

Property	Meaning	Example
Atomicity	Everything happens together or nothing happens	₹500 deducted from A, but not added to B? Cancel the whole transaction.
Consistency	Database remains correct before and after transaction	Total balance remains correct: ₹1000 + ₹2000 = ₹3000 → ₹500 + ₹2700 = ₹3200.
Isolation	One transaction doesn't affect another	T1 and T2 execute independently, no interference.
Durability	Once committed, it is permanent	Changes to A and B will remain even if the system crashes.

3.[D]Is this schedule: serializable? Conflict serializable? Or both explain your own answer?

A Schedule (a):



অপারেশন:

Step Transaction Operation Data Item

1	T1	Read	X
2	T2	Read	X
3	T1	Write	Y
4	T2	Write	Y
5	T1	Commit	
6	T2	Commit	

🔍 Step 1: Identify Conflicts

Conflict rules:

- **Read-Write (RW)** conflict on same data.
- **Write-Read (WR)** conflict on same data.
- **Write-Write (WW)** conflict on same data.

- ◆ **Read(X)** (T1) vs **Read(X)** (T2) →  No conflict (both are reads)
 - ◆ **Write(Y)** (T1) vs **Write(Y)** (T2) →  Conflict exists (Write-Write conflict)
-

Step 2: Build Precedence (conflict) graph

- T1 does **Write(Y)** before T2 does **Write(Y)** ⇒ draw edge: **T1 → T2**

Graph:

T1 → T2

- No cycle 
 - So, **conflict serializable**
 - And since conflict serializable ⇒ also **serializable**
-

 **Proved:**

- Schedule (a) is **conflict serializable** and **serializable**.

অসাধারণ! তুমি ঠিক ধরেছো!

তাহলে চল নতুন করে **Schedule (b)**-টা শুদ্ধভাবে দেখি, তারপর তার **conflict serializability** এবং **serializability** প্রমাণ করি।

 **Updated Schedule (b):**

Step	T1	T2
1	Read(A)	
2		Read(B)
3	Write(A)	

Step 1: Conflict Check

আমরা এখন দেখি দুইটা transaction কি এক ডেটার উপরে একসাথে কাজ করেছে?

- T1: Read(A), Write(A)
- T2: Read(B)

👉 লক্ষ্য করো:

T1 A নিয়ে কাজ করছে, আর T2 B নিয়ে কাজ করছে।

অর্থাৎ তারা আলাদা আলাদা ডেটা নিয়ে কাজ করছে — A ও B

Step 2 Precedence (Conflict) Graph

- কোনো common data item (A/B) নাই, তাই কোনো conflict নাই
- ফলে conflict graph হবে "শূন্য" — কোনো edge নাই

Graph:

(No edges, no cycle)

Conclusion:

- কোনো conflict নাই
 - ➡ তাই এটা conflict serializable
 - ➡ এবং serializable ও বটে

END Final Judgment (Updated b):

Schedule Conflict Serializable? Serializable?

(b) ✓ হ্যাঁ ✓ হ্যাঁ

তুমি চাইলে আমি দুইটা schedule এর জন্য conflict graph diagram ঐকে দেখাতে পারি —
চাইলেই বলো! 😊✍️